ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

MÔN HỌC: CSC14003 - NHẬP MÔN TRÍ TUỆ NHÂN TẠO

BÁO CÁO ĐỒ ÁN MÔN HỌC

BLOXORZ SOLVER

Giảng viên hướng dẫn: TS. Bùi Tiến Lên

Lớp: 24_22

Năm học: Học kỳ I, 2026 - 2027

Sinh viên thực hiện:

MSSV	Họ và tên	Email sinh viên
24120127	Nguyễn Ngọc Quang	24120127@student.hcmus.edu.vn
24120134	Lương Nhật Tân	24120134@student.hcmus.edu.vn
24120185	Nguyễn Gia Hưng	24120185@student.hcmus.edu.vn
24120202	Võ Nhật Minh	24120202@student.hcmus.edu.vn
24120205	Trần Văn Nguyên	24120205@student.hcmus.edu.vn

Nội dung

1 Giới thiệu đồ án **1**

2 Mô hình hóa và kiến trúc **2**

2.1 Mô hình tìm kiếm 2

2.2 Kiến trúc chương trình 3

2.3 Đầu ra và số đo 3

3 Tổng quan các thuật toán **4**

3.1 BFS và DFS 4

3.2 UCS và A* 4

3.3 Quy ước đọc các kết quả 4

4 Thuật toán Breadth-First Search (BFS) **5**

4.1 Ý tưởng 5

4.2 Mã giả 5

4.3 Cài đặt 5

4.4 Tính chất 6

5 Thực nghiệm BFS **7**

5.1 Thiết lập 7

5.2 Kết quả 7

5.3 Nhận xét 8

5.4 Kết luận 8

6 Thuật toán Depth-First Search (DFS) 9

6.1 Ý tưởng . . . 9

6.2 Mã giả . . . 9

6.3 Cài đặt . . . 9

6.4 Tính chất . . . 10

7 Thực nghiệm DFS 11

7.1 Thiết lập . . . 11

7.2 Kết quả . . . 11

7.3 Nhận xét . . . 12

7.4 Kết luận . . . 12

8 Tìm kiếm chi phí đều (UCS) 13

8.1 Mục tiêu . . . 13

8.2 Mô hình hóa bài toán tìm kiếm . . . 13

8.3 Hàm chi phí . . . 14

8.3.1 Công thức . . . 14

8.3.2 Lý do chọn các hệ số . . . 14

8.3.3 Tính chất cần cho UCS . . . 15

8.4 Thuật toán và cài đặt . . . 15

8.4.1 Ý tưởng . . . 15

8.4.2 Mã giả . . . 16

8.4.3 Cấu trúc dữ liệu . . . 16

8.4.4 Xử lý phần tử cũ và thời điểm kiểm tra đích . . . 16

8.4.5 Tính đúng đắn và độ phức tạp . . . 17

8.4.6 Cài đặt và cách chạy . . . 17

8.4.7 Đoạn mã nguồn cốt lõi . . . 17

8.5 Thực nghiệm . . . 18

8.5.1 Thiết lập và kết quả . . . 18

8.5.2 Nhận xét . . . 19

8.6 Đối chiếu các phần thực hiện . . . 20
8.7 Kết luận . . . 20

9 Thuật toán A* và hàm ước lượng 21

9.1 Phạm vi thực hiện . . . 21
9.1.1 Đầu vào và đầu ra . . . 21
9.1.2 Ranh giới giữa các thành phần . . . 21
9.2 Mô hình chi phí và cấu trúc dữ liệu . . . 22
9.2.1 Quy tắc tính chi phí dùng chung . . . 22
9.2.2 Cấu trúc tìm kiếm . . . 23
9.3 Thuật toán A* và cách tích hợp . . . 23
9.3.1 Hàm ưu tiên . . . 23
9.3.2 Mã giả . . . 23
9.3.3 Loại phần tử cũ, mở lại trạng thái và điều kiện tối ưu . . . 24
9.3.4 Cách gọi từ chương trình . . . 24
9.4 Thiết kế và chứng minh hàm ước lượng . . . 24
9.4.1 Vì sao không dùng Manhattan cộng chi phí tư thế . . . 24
9.4.2 Mô hình điểm và các điểm mốc . . . 25
9.4.3 Chứng minh tính nhất quán và tính chấp nhận được . . . 25
9.4.4 Chi phí tính toán và giới hạn . . . 26
9.5 Thực nghiệm và đánh giá . . . 26
9.5.1 Mục tiêu và cách bố trí thực nghiệm . . . 26
9.5.2 Cách đo thời gian và bộ nhớ . . . 27
9.5.3 Kiểm chứng lời giải và số nút mở rộng . . . 27
9.5.4 Biểu đồ và nhận xét . . . 28

10 Đánh giá và phân công 29

10.1 Tổng hợp kết quả . . . 29
10.2 So sánh định lượng trên cùng bộ level . . . 29
10.3 Kiểm chứng và giới hạn . . . 30
10.4 Phân công và tự đánh giá . . . 30

11 Phụ lục: Công khai việc sử dụng công cụ AI 32

1. Giới thiệu đồ án

Đồ án xây dựng trò Bloxorz có giao diện đồ họa, luật chơi và bộ giải tự động bằng bốn phương pháp tìm kiếm: Breadth-First Search (BFS), Depth-First Search (DFS), Uniform-Cost Search (UCS) và A*. Chương trình dùng chung mô hình màn chơi và luật chuyển trạng thái để người chơi có thể chơi thủ công hoặc phát lại lời giải của solver.

Các mục tiêu chính:

- Mô hình hóa chính xác tư thế khối, nước đi, điều kiện thắng và các ô đặc biệt theo đề bài.
- Cài đặt bốn thuật toán tìm kiếm bằng Python, không giao việc tìm kiếm cho thư viện bên ngoài.
- Đo thời gian tìm kiếm, bộ nhớ đỉnh, số nút mở rộng và độ dài lời giải; kiểm tra lời giải bằng cách phát lại qua luật trò chơi.
- Trình bày thiết kế, kết quả và giới hạn của từng thuật toán trong các mục riêng của báo cáo.

2. Mô hình hóa và kiến trúc

2.1. Mô hình tìm kiếm

- **Trạng thái:** `GameState` chứa `BlockState` và `bridge_mask`. `BlockState` lưu tọa độ neo, tư thế, cờ tách khối, vị trí hai khối lập phương và chỉ số khối đang điều khiển. `bridge_mask` phân biệt các cấu hình cầu.
- **Trạng thái đầu:** trạng thái khối ban đầu của màn cùng với mặt nạ cầu ban đầu.
- **Hành động:** `U`, `D`, `L`, `R`; khi khối đã tách, hành động `SPACE` đổi khối lập phương được điều khiển.
- **Hàm chuyển:** `get_successors(state, level)` chỉ sinh các hành động hợp lệ. Luật lõi loại nước đi ra ngoài bàn, rơi vào ô trống hoặc cầu đóng, và đứng trên ô dễ vỡ; đồng thời cập nhật công tắc, cầu, trạng thái tách và ghép khối.
- **Kiểm tra đích:** `is_goal(state, level)` chỉ đúng khi khối chưa tách, đứng thẳng và ở ô đích.

Vì `GameState` là bất biến và có thể băm, solver dùng toàn bộ trạng thái làm khóa để tránh lặp. Hai cấu hình có cùng tọa độ nhưng khác tư thế, trạng thái cầu hoặc khối đang điều khiển là hai trạng thái khác nhau. Không gian trạng thái hữu hạn do mỗi màn có hữu hạn ô và hữu hạn cầu.

2.2. Kiến trúc chương trình

Bảng 1: Các thành phần chính được dùng trong đồ án.

Thành phần	Trách nhiệm
`core/interfaces.py`	Định nghĩa `Level`, `GameState`, `BlockState` và `SolverResult`.
`core/game_rules.py`	Tính nước đi hợp lệ, sinh successor và kiểm tra điều kiện thắng.
`solvers/common.py`	Giao diện solver, hàm dựng đường đi, `StepCost` và đo hiệu năng.
`solvers/BFS.py`, `solvers/DFS.py`, `solvers/UCS.py`, `solvers/astar.py`	Cài đặt bốn phương pháp tìm kiếm.
`ui/` và `rendering/`	Giao diện người chơi, điều khiển, hoạt ảnh và hiển thị bàn cờ.
`levels/`	Đọc cấu hình JSON cho các màn chơi.

2.3. Đầu ra và số đo

Mỗi solver trả về `SolverResult`, gồm đường đi, trạng thái thành công, tổng chi phí, độ dài lời giải và số nút mở rộng. `run_solver()` bổ sung thời gian tìm kiếm bằng `time.perf_counter()` và bộ nhớ cấp phát Python đỉnh bằng `tracemalloc` khi chạy qua giao diện. Trong benchmark báo cáo, thời gian và bộ nhớ được đo ở hai lượt riêng để không làm `tracemalloc` làm chậm phép đo thời gian; quy trình này được áp dụng đồng đều cho bốn solver và mô tả trong các mục thực nghiệm.

Lời giải thành công được phát lại từ trạng thái đầu bằng `apply_move`. Mỗi hành động phải hợp lệ, trạng thái cuối phải thỏa `is_goal`, và với UCS và A* có tổng chi phí phát lại phải khớp chi phí solver báo cáo.

3. Tổng quan các thuật toán

3.1. BFS và DFS

BFS sử dụng hàng đợi FIFO, duyệt theo từng lớp độ sâu và bảo đảm lời giải có số hành động ít nhất khi mỗi hành động được tính bằng một bước. DFS sử dụng ngăn xếp LIFO, đi sâu theo một nhánh trước và có thể trả về lời giải dài hơn; tập trạng thái đã thăm giúp tránh chu trình. Hai thuật toán này tối ưu theo số hành động chứ không theo `StepCost`.

3.2. UCS và A*

UCS mở rộng trạng thái có chi phí tích lũy $g(n)$ thấp nhất. A* ưu tiên $f(n) = g(n) + h(n)$, trong đó $h(n)$ là cận dưới ước lượng chi phí còn lại tới đích. Cả hai dùng chính sách `StepCost` chung; UCS dùng hàng đợi ưu tiên theo g, còn A* theo f. Tính tối ưu của A* phụ thuộc vào tính chấp nhận được của heuristic đối với chính sách chi phí đang dùng.

3.3. Quy ước đọc các kết quả

Các mục BFS, DFS, UCS và A* trình bày riêng mã giả, cấu trúc dữ liệu, đặc tính, kết quả và giới hạn. Cả bốn cùng dùng 11 level hiện hành, trạng thái đầu của từng level và một quy trình đo thống nhất trong `typst/collect_all_results.py`. Mục 8 tổng hợp các chỉ số trên cùng bộ test để việc so sánh có cùng điều kiện.

Đường đi và kết quả trong báo cáo được xem là hợp lệ khi phát lại qua luật trò chơi. `SPACE` được tính là một hành động; với UCS/A*, mọi hành động cũng có chi phí dương theo `StepCost`. BFS tối ưu số hành động, trong khi UCS/A* tối ưu tổng `StepCost`; DFS không bảo đảm tối ưu.

4. Thuật toán Breadth-First Search (BFS)

4.1. Ý tưởng

BFS duyệt các trạng thái theo từng lớp độ sâu: mọi trạng thái cách trạng thái đầu k hành động được mở rộng trước các trạng thái cách $k+1$ hành động. Cài đặt dùng hàng đợi FIFO (`collections.deque`) và từ điển `came_from` làm tập đã thăm.

Bài kiểm tra đích được thực hiện ngay khi sinh trạng thái con, thay vì lúc lấy ra khỏi hàng đợi. Cách này giảm số nút phải mở rộng mà vẫn giữ được tính tối ưu, vì mọi hành động đều có chi phí 1.

4.2. Mã giả

Thuật toán 1: Breadth-First Search (BFS) cho Bloxorz

Đầu vào: màn chơi `level`, trạng thái đầu `start`
Đầu ra: dãy hành động ngắn nhất tới đích, hoặc vô nghiệm

nếu `is_goal(start)` thì trả về dãy rỗng
`came_from[start] = None`; `queue = [start]`; `expanded = 0`
trong khi `queue` không rỗng:
    `state = queue.popleft()`; `expanded += 1`
    với mỗi `(action, nxt)` trong `get_successors(state, level)`:
        nếu `nxt` đã có trong `came_from` thì bỏ qua
        `came_from[nxt] = (state, action)`
        nếu `is_goal(nxt)` thì trả về đường đi dựng lại từ `came_from`
        `queue.append(nxt)`
trả về vô nghiệm (hàng đợi cạn)

4.3. Cài đặt

Vòng lặp chính trong `solvers/BFS.py`:

```
    came_from = {start: None}
    queue = deque([start])
    expanded = 0

    while queue:
        state = queue.popleft()
        expanded += 1
        for action, nxt in get_successors(state, level):
```

```
            if nxt in came_from:
                continue
            came_from[nxt] = (state, action)
            if is_goal(nxt, level):
                return make_result(reconstruct_path(came_from, nxt),
                expanded)
            queue.append(nxt)
```

Nếu trạng thái đầu đã là đích, solver trả về dãy rỗng. Nếu hàng đợi cạn mà chưa gặp đích, solver trả về `path=None`, tức màn chơi vô nghiệm. Đường đi được dựng lại bằng `reconstruct_path` trong `solvers/common.py`, dùng chung với các solver khác.

4.4. Tính chất

- **Đầy đủ**: nếu có lời giải thì BFS tìm được, vì không gian trạng thái hữu hạn.
- **Tối ưu**: vì mọi hành động có chi phí 1, lời giải tìm được có số hành động ít nhất.
- **Độ phức tạp**: với hệ số nhánh $b \leq 4$ (hoặc $b \leq 5$ khi khối bị tách) và độ sâu lời giải d, thời gian và bộ nhớ đều ở mức $O(b^d)$. Nhờ tập trạng thái đã thăm, cả hai còn bị chặn trên bởi số trạng thái đạt được $|S|$.
- **Nhược điểm chính**: phải giữ toàn bộ biên tìm kiếm và mọi trạng thái đã thăm trong bộ nhớ, nên bộ nhớ tăng nhanh theo kích thước không gian trạng thái.

Ghi chú

Số nút mở rộng thực tế (xem phần thực nghiệm) nhỏ hơn rất nhiều so với b^d, vì nhiều chuỗi hành động khác nhau dẫn tới cùng một trạng thái và bị loại bởi `came_from`.

5. Thực nghiệm BFS

5.1. Thiết lập

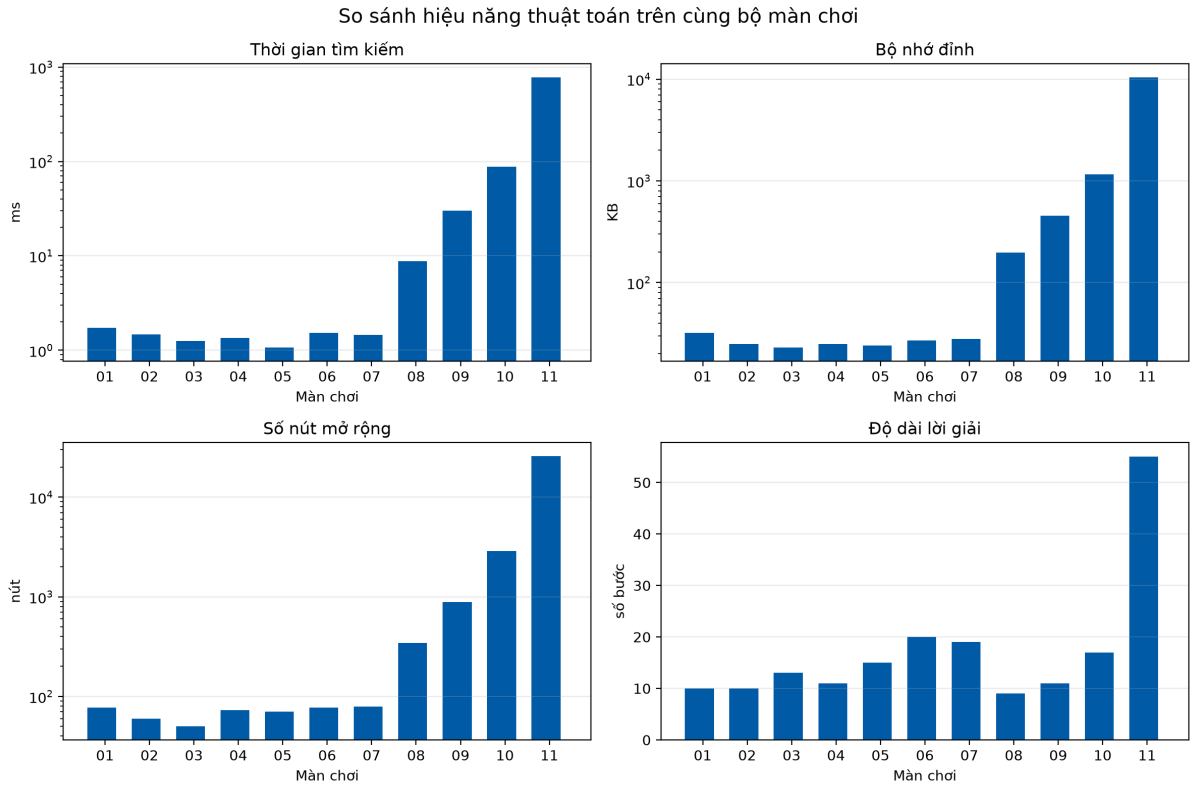
BFS được chạy từ trạng thái đầu của từng level trong cùng bộ dữ liệu 11 màn với DFS, UCS và A*. Các nhóm level được phân theo cơ chế hiện diện: cơ bản, có ô dễ vỡ, có công tắc/cầu, hoặc kết hợp ô dễ vỡ với công tắc. Bộ nhớ đỉnh được đo một lần riêng cho từng solver và level. Mọi lời giải được phát lại qua luật trò chơi để kiểm tra tính hợp lệ và đích đến.

Thời gian được đo bằng `perf_counter()` quanh `solve()` khi `tracemalloc` tắt, áp dụng như nhau cho cả bốn solver. Đo bộ nhớ dùng một lượt riêng với `tracemalloc`; số đo là đỉnh cấp phát, không phải toàn bộ RAM tiến trình. Nút mở rộng được đếm theo quy ước riêng của BFS: trạng thái lấy khỏi hàng đợi; trạng thái đích được nhận lúc sinh nên không tính vào số nút mở rộng.

5.2. Kết quả

Bảng 2: Kết quả BFS trên bộ level hiện hành; thời gian là trung vị 3 lượt đo, bộ nhớ đo riêng một lượt.

Màn	Nhóm	Giải được	Thời gian trung vị (ms)	ĐLC thời gian (ms)	Bộ nhớ đỉnh (KB)	Nút mở rộng	Độ dài lời giải
level_01	Cơ bản	Có	1.734	0.031	32	77	10
level_02	Công tắc/ cầu	Có	1.476	0.177	25	60	10
level_03	Công tắc/ cầu	Có	1.26	0.16	23	50	13
level_04	Ô dễ vỡ	Có	1.358	0.298	25	73	11
level_05	Ô dễ vỡ	Có	1.071	0.047	24	70	15
level_06	Công tắc/ cầu	Có	1.525	0.334	27	77	20
level_07	Công tắc/ cầu	Có	1.452	0.025	28	79	19
level_08	Kết hợp	Có	8.809	0.411	198	343	9
level_09	Kết hợp	Có	30.276	3.566	453	887	11
level_10	Kết hợp	Có	88.331	2.495	1165	2864	17
level_11	Kết hợp	Có	779.818	22.496	10456	25815	55



Hình 1: Thời gian, bộ nhớ đỉnh, số nút mở rộng và độ dài lời giải của BFS trên từng level.

5.3. Nhận xét

BFS tối ưu số hành động vì mỗi nước đi có cùng chi phí một bước. Bảng ghi kết quả đo mới cho từng level, thay cho các số liệu cũ không còn đồng bộ với level hiện tại. Những level có nhiều trạng thái hợp lệ hơn có thể làm hàng đợi và bộ nhớ tăng; kết quả cụ thể cần được đọc cùng nhóm cơ chế và so sánh trong bảng tổng hợp bốn thuật toán.

5.4. Kết luận

BFS phù hợp khi mục tiêu là lời giải có ít hành động nhất. Đổi lại, nó không tối ưu theo `StepCost`, và mức dùng bộ nhớ phụ thuộc lượng trạng thái được giữ trong hàng đợi và tập đã thăm.

6. Thuật toán Depth-First Search (DFS)

6.1. Ý tưởng

DFS đi sâu theo một nhánh trước, chỉ quay lui khi hết đường. Cài đặt dùng ngăn xếp LIFO ở dạng lặp để tránh tràn ngăn xếp gọi hàm ở các màn có nhiều trạng thái.

Một trạng thái được đánh dấu đã thăm khi được lấy ra khỏi ngăn xếp, đồng thời cạnh cha được ghi vào `came_from` ngay lúc đó. Nhờ vậy đường đi trả về chính là nhánh DFS thật sự, không bị lẫn với các nhánh khác. Các trạng thái con được đẩy theo thứ tự ngược để hành động đầu tiên trong `get_successors` được thử trước, giúp kết quả tất định.

6.2. Mã giả

Thuật toán 2: Depth-First Search (DFS) cho Bloxorz

Đầu vào: màn chơi `level`, trạng thái đầu `start`
Đầu ra: một dãy hành động hợp lệ tới đích, hoặc vô nghiệm

```
stack = [(start, None, None)]; expanded = 0
trong khi stack không rỗng:
    (state, parent, action) = stack.pop()
    nếu state đã có trong came_from thì bỏ qua
    came_from[state] = (parent, action)
    nếu is_goal(state) thì trả về đường đi dựng lại từ came_from
    expanded += 1
    với mỗi (act, nxt) trong reversed(get_successors(state, level)):
        nếu nxt chưa có trong came_from thì stack.push((nxt, state, act))
trả về vô nghiệm (ngăn xếp cạn)
```

6.3. Cài đặt

Vòng lặp chính trong `solvers/DFS.py`:

```python
    while stack:
        state, parent, action = stack.pop()
        if state in came_from:
            continue
        came_from[state] = None if parent is None else (parent, action)

        if is_goal(state, level):
            return make_result(reconstruct_path(came_from, state),
            expanded)
```

```
9
10          expanded += 1
11          for act, nxt in reversed(get_successors(state, level)):
12              if nxt not in came_from:
13                  stack.append((nxt, state, act))
```

Bài kiểm tra đích được thực hiện khi lấy trạng thái ra khỏi ngăn xếp, và nút đích không được tính vào số nút mở rộng.

6.4. Tính chất

- **Đầy đủ**: nhờ tập trạng thái đã thăm và không gian trạng thái hữu hạn, DFS không bị kẹt trong chu trình và tìm được lời giải nếu có.
- **Không tối ưu**: lời giải đầu tiên tìm được có thể dài hơn nhiều so với lời giải ngắn nhất.
- **Phụ thuộc thứ tự hành động**: kết quả phụ thuộc vào thứ tự `U, D, L, R` mà `get_successors` trả về và vào hình dạng của màn chơi.
- **Bộ nhớ**: về lý thuyết, DFS dạng cây chỉ cần $O(bm)$ với m là độ sâu tối đa. Tuy nhiên bản cài đặt của nhóm vẫn lưu `came_from` cho mọi trạng thái đã thăm (để chống chu trình và dựng đường đi), nên bộ nhớ vẫn tăng theo số trạng thái đã khám phá.

Cảnh báo / Lưu ý quan trọng

DFS chỉ cho một lời giải hợp lệ, không bảo đảm ngắn nhất. Nếu cần đường đi ngắn hơn mà vẫn giữ tinh thần tìm kiếm theo chiều sâu, có thể dùng Iterative Deepening Search (IDS) thay cho DFS, đề bài cho phép lựa chọn này.

7. Thực nghiệm DFS

7.1. Thiết lập

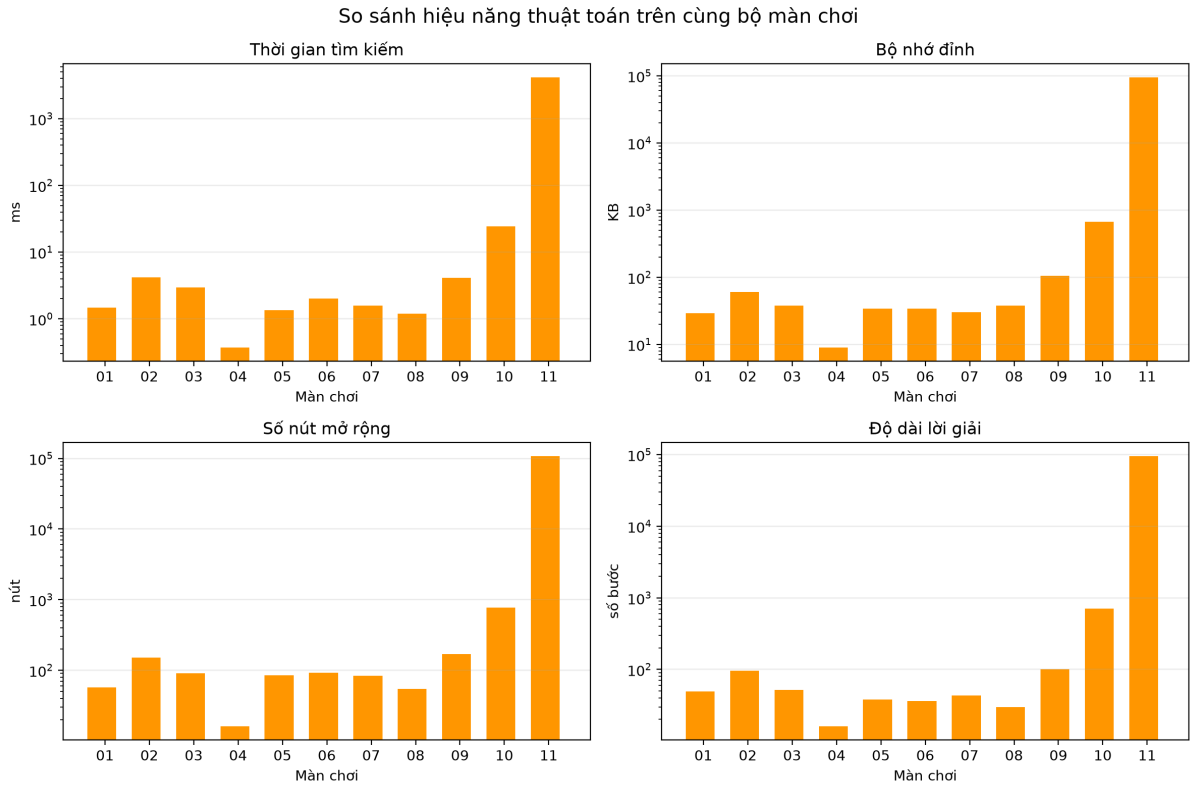
DFS được chạy trên cùng 11 level và cùng trạng thái đầu như ba solver còn lại. Các level được nhóm theo ô dễ vỡ và công tắc/cầu. Bộ nhớ đỉnh được đo một lần riêng cho từng solver và level. Mỗi đường đi được phát lại để xác minh từng nước đi hợp lệ và trạng thái cuối đúng đích.

Thời gian đo bằng `perf_counter()` khi `tracemalloc` tắt, giống ba solver còn lại. Bộ nhớ được đo trong lượt riêng với `tracemalloc`, là đỉnh cấp phát chứ không phải toàn bộ RAM tiến trình. Nút mở rộng của DFS là số trạng thái được lấy khỏi stack và thực sự mở rộng; trạng thái đích không được tính.

7.2. Kết quả

Bảng 3: Kết quả DFS trên bộ level hiện hành; thời gian là trung vị 3 lượt đo, bộ nhớ đo riêng một lượt.

Màn	Nhóm	Giải được	Thời gian trung vị (ms)	ĐLC thời gian (ms)	Bộ nhớ đỉnh (KB)	Nút mở rộng	Độ dài lời giải
level_01	Cơ bản	Có	1.462	0.242	29	57	49
level_02	Công tắc/ cầu	Có	4.156	0.938	60	152	96
level_03	Công tắc/ cầu	Có	2.95	0.098	38	90	52
level_04	Ô dễ vỡ	Có	0.373	0.06	9	16	16
level_05	Ô dễ vỡ	Có	1.346	0.277	34	85	38
level_06	Công tắc/ cầu	Có	2.004	0.306	34	92	36
level_07	Công tắc/ cầu	Có	1.583	0.007	30	84	43
level_08	Kết hợp	Có	1.195	0.069	38	54	30
level_09	Kết hợp	Có	4.149	0.132	105	168	101
level_10	Kết hợp	Có	24.56	2.954	673	769	710
level_11	Kết hợp	Có	4173.356	151.826	95141	107660	95212



Hình 2: Thời gian, bộ nhớ đỉnh, số nút mở rộng và độ dài lời giải của DFS trên từng level.

7.3. Nhận xét

DFS trả về đường hợp lệ đầu tiên theo thứ tự duyệt, không bảo đảm số bước ít nhất. Vì vậy độ dài lời giải có thể chênh đáng kể so với BFS; bảng và biểu đồ mới cho thấy mức chênh trên cùng các level, thay vì dùng số liệu của các phiên bản màn trước. Kết quả cũng phụ thuộc thứ tự successor và cấu trúc từng màn.

Cảnh báo / Lưu ý quan trọng

DFS chỉ cho một lời giải hợp lệ, không bảo đảm ngắn nhất, và hiệu năng không ổn định giữa các màn. Vì vậy không nên dùng DFS khi cần đường đi ngắn.

7.4. Kết luận

DFS phù hợp khi chỉ cần tìm một lời giải hợp lệ và không yêu cầu tối ưu số bước. Tính đầy đủ trong bộ test này không đồng nghĩa với hiệu năng hay chất lượng đường đi ổn định trên mọi màn.

8. Tìm kiếm chi phí đều (UCS)

8.1. Mục tiêu

Phần này trình bày thuật toán tìm kiếm chi phí đều (Uniform-Cost Search, UCS) dùng để giải các màn Bloxorz. UCS tìm dãy hành động đưa khối từ trạng thái bắt đầu tới đích sao cho tổng chi phí các bước là nhỏ nhất. Khác với BFS chỉ đếm số bước, UCS dùng hàm chi phí `StepCost` dùng chung trong `solvers/common.py`, nên có thể ưu tiên đường đi ít rủi ro hơn dù dài hơn.

Mã nguồn nằm trong `solvers/UCS.py`, gồm lớp `UCSSolver` kế thừa lớp trừu tượng `Solver` và chính sách chi phí `StepCost`. Lớp này chỉ chứa thuật toán tìm kiếm; luật chơi lấy từ `core/game_rules.py`, chi phí lấy từ `solvers/common.py`.

8.2. Mô hình hóa bài toán tìm kiếm

- **Trạng thái:** một `GameState` gồm `BlockState` (vị trí ô mốc `(r, c)`, tư thế đứng / nằm ngang / nằm dọc, cờ đã tách, vị trí hai khối lập phương và khối đang điều khiển) cùng `bridge_mask`, mặt nạ bit trạng thái đóng/mở của từng cầu. Hai trạng thái cùng vị trí khối nhưng khác trạng thái cầu là hai trạng thái khác nhau.
- **Trạng thái bắt đầu:** `GameState(level.start, level.initial_bridge_mask)`.
- **Hành động:** `U`, `D`, `L`, `R` lăn khối theo bốn hướng. Hành động `SPACE` chỉ hợp lệ khi khối đã bị tách bởi công tắc tách, dùng để đổi khối lập phương đang điều khiển.
- **Hàm chuyển trạng thái:** `get_successors(state, level)` trả về các cặp `(action, next_state)` hợp lệ. Phần lõi đã loại các nước làm khối rơi ra ngoài bản đồ, vào ô trống hoặc cầu đang đóng, và các nước đứng thẳng trên ô dễ vỡ; đồng thời xử lý công tắc, tách và ghép khối.
- **Kiểm tra đích:** `is_goal(state, level)` đúng khi khối chưa tách, đang đứng và nằm trên ô đích. Một khối lập phương đơn lẻ trên ô đích không được tính là thắng.
- **Chi phí bước:** `step_cost(state, action, next_state, level)`, luôn dương .

Mỗi màn có số ô hữu hạn và số cầu hữu hạn, nên không gian trạng thái hữu hạn.

8.3. Hàm chi phí

8.3.1. Công thức

Gọi s là trạng thái trước, a là hành động và t là trạng thái sau. UCS dùng chi phí của lớp `StepCost`:

$$c(s, a, t) = e(s, a, t) + \sum_{x \in C(t)} p(x) + b(s, t). \tag{1}$$

Trong đó e là công sức của chính nước đi, $C(t)$ là tập ô mà khối chiếm sau khi đi, $p(x)$ là mức phạt rủi ro của ô x và $b(s, t)$ là mức phạt khi nước đi làm thay đổi trạng thái cầu.

Bảng 4: Công sức của từng loại nước đi.

Loại nước đi	Công sức e
Đứng → nằm (lật đổ)	1.0
Nằm → đứng (dựng khối lên)	1.5
Nằm → nằm (lăn theo cạnh dài)	1.2
Di chuyển một khối lập phương khi đã tách	0.8
`SPACE`: đổi khối lập phương đang điều khiển	0.3

Bảng 5: Mức phạt rủi ro.

Tình huống	Mức phạt
Mỗi ô dễ vỡ bị chiếm	+0.5
Mỗi ô cầu bị chiếm	+0.3
Mỗi ô bị chiếm có ô kề (4 hướng) là khoảng trống	+0.1
Nước đi làm thay đổi `bridge_mask`	+0.5

8.3.2. Lý do chọn các hệ số

Đề bài không quy định chi phí; nếu mọi nước đi có chi phí 1 thì UCS cho thứ tự tìm kiếm giống BFS. Các hệ số được chọn theo hai ý:

- **Công sức:** dựng khối lên phải nâng khối nên đắt nhất; lăn khi đang nằm tốn hơn lật đổ một chút; một khối lập phương chỉ nặng bằng nửa khối nên rẻ hơn; `SPACE` không di chuyển khối nhưng vẫn phải có chi phí dương để thuật toán không lặp `SPACE` vô hạn.
- **Rủi ro:** ô dễ vỡ, cầu và ô sát mép là những vị trí dễ dẫn tới thua; nước bấm công tắc thay đổi bản đồ nên được tính là thao tác tốn kém.

Mức phạt rủi ro nhỏ hơn hẳn công sức một nước đi. Vì vậy UCS vẫn ưu tiên đường ngắn, và chỉ chọn đường an toàn hơn khi các đường có độ dài gần nhau. Đây là mô hình định lượng do nhóm đặt ra, không phải phép đo vật lý.

8.3.3. Tính chất cần cho UCS

Mọi mức phạt đều không âm, nên với chính sách mặc định:

$$c(s, a, t) \geq \min(1.0, 1.5, 1.2, 0.8, 0.3) = 0.3 > 0. \tag{2}$$

Chi phí dương là điều kiện để UCS bảo đảm lời giải tối ưu (mục sau).

Ghi chú

Mức phạt của mỗi ô chỉ phụ thuộc bản đồ, nên `_precompute_cell_penalty` tính sẵn một bảng `ô → mức phạt` một lần ở đầu mỗi lần giải. Khi đó `step_cost` chỉ cộng công sức với vài phép tra bảng.

8.4. Thuật toán và cài đặt

8.4.1. Ý tưởng

UCS luôn mở rộng trạng thái có chi phí tích lũy nhỏ nhất:

$$g(s) = \sum_{i=1}^{k} c(s_{i-1}, a_i, s_i), \quad s_0 = \text{start}, s_k = s. \tag{3}$$

Hàng đợi ưu tiên sắp theo g. UCS là trường hợp riêng của A* với hàm ước lượng $h = 0$, và trùng với BFS khi mọi bước có cùng chi phí.

8.4.2. Mã giả

Thuật toán 3: Tìm kiếm chi phí đều trên đồ thị trạng thái Bloxorz

Đầu vào: Màn chơi, trạng thái bắt đầu, hàm chi phí dương `step_cost`.
Đầu ra: Dãy hành động có tổng chi phí nhỏ nhất, hoặc không có lời giải.

Tính sẵn mức phạt của các ô; đặt `best_cost[start] = 0`, `came_from[start] = None`.
Đưa `(0, counter, start)` vào hàng đợi ưu tiên `frontier`.
Khi `frontier` còn phần tử: lấy phần tử `(g, _, state)` có `g` nhỏ nhất.
Nếu `state` đã mở rộng hoặc `g > best_cost[state]`, bỏ qua (phần tử cũ).
Đánh dấu `state` đã mở rộng, tăng `expanded_nodes`.
Nếu `is_goal(state)`: dựng đường đi từ `came_from`, trả về chi phí `g`.
Với mỗi `(action, next)` trong `get_successors(state)` mà `next` chưa mở rộng:
    `new_g = g + step_cost(state, action, next)`.
    Nếu `new_g < best_cost[next]`: cập nhật `best_cost`, `came_from`, đưa `(new_g, counter, next)` vào `frontier`.
Nếu `frontier` cạn, trả về không có lời giải.

8.4.3. Cấu trúc dữ liệu

Bảng 6: Cấu trúc dữ liệu của `UCSSolver.solve`.

Biến	Vai trò
`frontier`	Đống cực tiểu (`heapq`) chứa `(g, counter, state)`.
`counter`	Bộ đếm tăng dần, phân giải hòa `g` theo thứ tự đưa vào, tránh so sánh hai `GameState`.
`best_cost`	Chi phí nhỏ nhất đã biết để tới mỗi trạng thái.
`came_from`	`trạng thái → (trạng thái trước, hành động)`, dùng để dựng lời giải.
`expanded`	Tập trạng thái đã mở rộng; mỗi trạng thái chỉ mở rộng một lần.

`GameState` là dataclass bất biến (`frozen=True`) nên dùng trực tiếp làm khóa của `dict` và phần tử của `set`. Khóa gồm cả `bridge_mask`, nên các cấu hình cầu khác nhau được phân biệt đúng.

8.4.4. Xử lý phần tử cũ và thời điểm kiểm tra đích

`heapq` không có thao tác giảm khóa. Khi tìm được đường rẻ hơn tới một trạng thái, thuật toán đưa thêm một bản mới vào hàng đợi; bản cũ có `g` lớn hơn `best_cost` sẽ bị bỏ qua khi được lấy ra và không được tính vào `expanded_nodes`.

Đích được kiểm tra khi trạng thái được **lấy ra** khỏi hàng đợi, không phải khi vừa sinh ra. Lúc lấy ra, mọi phần tử còn lại có g không nhỏ hơn, và mọi chi phí đều dương, nên không còn đường nào rẻ hơn tới trạng thái đó.

8.4.5. Tính đúng đắn và độ phức tạp

- **Đầy đủ:** không gian trạng thái hữu hạn và mỗi trạng thái chỉ mở rộng một lần, nên nếu có lời giải thì UCS tìm ra.
- **Tối ưu:** với $c \geq 0.3 > 0$, trạng thái được lấy ra lần đầu có g bằng chi phí đường đi nhỏ nhất (lập luận kiểu Dijkstra). Áp dụng cho trạng thái đích, lời giải trả về có tổng chi phí nhỏ nhất.
- **Thời gian:** $O(E \log E)$ với E là số cạnh đã xét; mỗi trạng thái có tối đa 5 trạng thái kế tiếp nên $E \leq 5N$, N là số trạng thái tới được.
- **Bộ nhớ:** $O(N + E)$ cho `best_cost`, `came_from`, `expanded` và hàng đợi (có thể chứa phần tử cũ).

8.4.6. Cài đặt và cách chạy

`UCSSolver` kế thừa `StepCost` để lấy hàm chi phí và `Solver` để có giao diện chung với các thuật toán khác. Đường đi được dựng bằng hàm dùng chung `reconstruct_path`. Kết quả trả về là `SolverResult`:

- Tìm được: `path`, `solution_length = len(path)`, `cost = g`, `success = True`.
- Không tìm được: `path = None`, `solution_length = -1`, `cost = inf`, `success = False`.

`search_time_ms` và `peak_memory_kb` do `run_solver` điền. Giao diện (`ui/dependencies.py`) trả về `UCSSolver` khi người dùng chọn UCS. Có thể chạy độc lập từ thư mục gốc dự án:

```
python3 -m solvers.UCS          # tất cả các màn
python3 -m solvers.UCS 1 8 10   # chỉ các màn 1, 8, 10
```

8.4.7. Đoạn mã nguồn cốt lõi

Đoạn mã dưới đây thể hiện bước cập nhật chi phí và đưa trạng thái vào hàng đợi trong `solvers/UCS.py`:

```
for action, next_state in get_successors(state, level):
    if next_state in expanded:
        continue
    new_g = g + self.step_cost(state, action, next_state, level)
    if new_g < best_cost.get(next_state, float("inf")):
```

```
            best_cost[next_state] = new_g
            came_from[next_state] = (state, action)
            heapq.heappush(frontier, (new_g, next(counter), next_state))
```

Điều kiện cập nhật dùng tổng chi phí, không dùng số hành động. Vì vậy thuật toán có thể chọn đường nhiều bước hơn nếu tổng chi phí thấp hơn.

8.5. Thực nghiệm

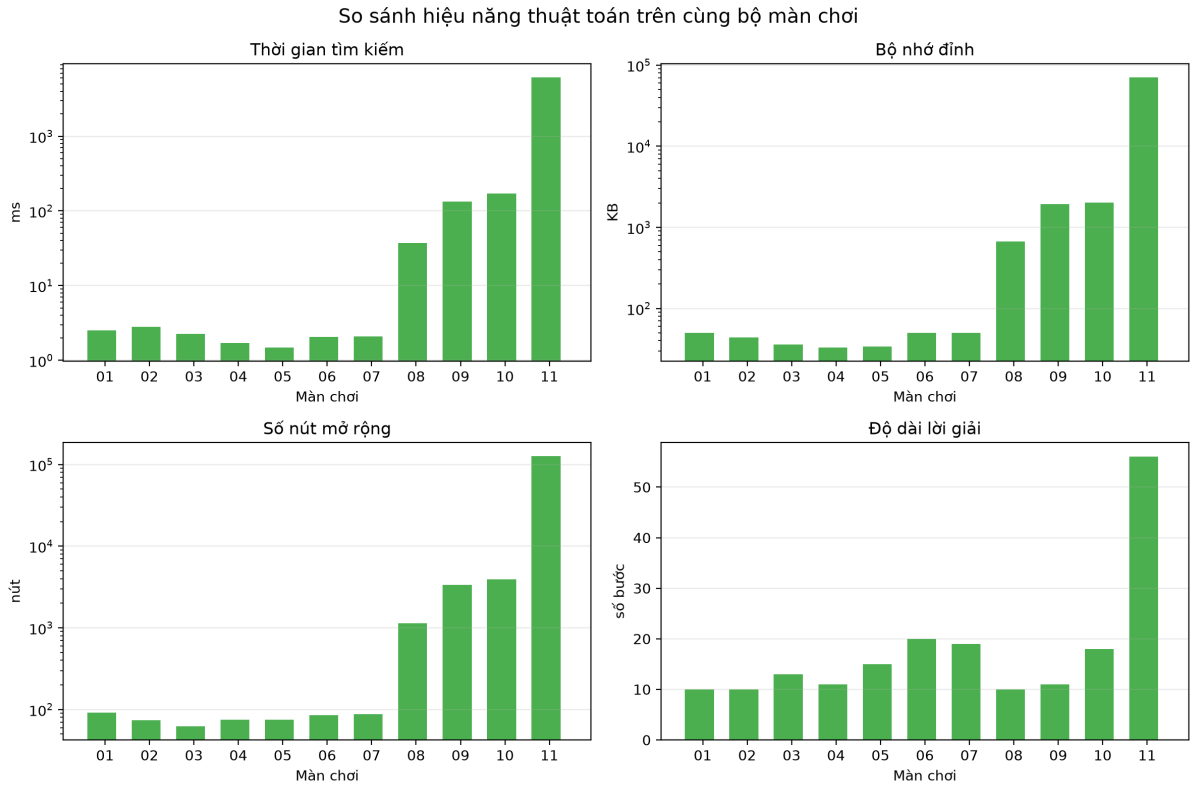
8.5.1. Thiết lập và kết quả

UCS được đo trên cùng 11 level và từ cùng trạng thái đầu như BFS, DFS và A*. Bộ nhớ đỉnh được đo trong một lượt riêng. Lời giải được phát lại từng bước; trạng thái đích, độ dài và tổng `StepCost` tính lại phải khớp kết quả trả về. Chi phí tối ưu của UCS được đối chiếu thêm với A* trên từng level.

Thời gian được đo bằng `perf_counter()` khi `tracemalloc` tắt; bộ nhớ được đo bằng `tracemalloc` riêng. Hai phép đo này dùng cùng quy trình cho cả bốn thuật toán; số đo bộ nhớ là đỉnh cấp phát, không phải tổng RAM tiến trình.

Bảng 7: Kết quả UCS trên level hiện hành; thời gian là trung vị 3 lượt đo, bộ nhớ đo riêng một lượt.

Màn	Nhóm	Giải được	Bước	Chi phí	Thời gian trung vị (ms)	ĐLC thời gian (ms)	Bộ nhớ đỉnh (KB)	Nút mở rộng
level_01	Cơ bản	Có	10	13.3	2.511	0.312	50	91
level_02	Công tắc/ cầu	Có	10	14.4	2.822	0.232	44	74
level_03	Công tắc/ cầu	Có	13	18.5	2.258	0.049	36	62
level_04	Ô dễ vỡ	Có	11	16.1	1.697	0.105	33	75
level_05	Ô dễ vỡ	Có	15	23.2	1.486	0.024	34	75
level_06	Công tắc/ cầu	Có	20	29	2.066	0.083	50	85
level_07	Công tắc/ cầu	Có	19	27.1	2.097	0.038	50	88
level_08	Kết hợp	Có	10	11	37.384	2.11	669	1144
level_09	Kết hợp	Có	11	15.3	132.694	26.702	1923	3354
level_10	Kết hợp	Có	18	24.5	169.912	4.952	2032	3925
level_11	Kết hợp	Có	56	81.16	160.605	153.101	70903	126688



Hình 3: Thời gian, bộ nhớ đỉnh, số nút mở rộng và độ dài lời giải của UCS trên từng level.

8.5.2. Nhận xét

UCS tối ưu tổng `StepCost`, không tối ưu số bước. Do đó số bước trong bảng có thể khác BFS khi lời giải nhiều bước có chi phí thấp hơn. So sánh thời gian và bộ nhớ với solver khác cần dùng cùng các level, trạng thái đầu và cách đo; bảng tổng hợp và biểu đồ ở phần đánh giá dùng đúng bộ dữ liệu chung này.

Hạn chế

UCS không biết đích nằm ở đâu nên lan đều theo mọi hướng. Để giảm số nút cần một hàm ước lượng chấp nhận được, tức là chuyển sang A*. Thuật toán cũng chưa có giới hạn thời gian hay số nút; trong giao diện, việc hủy được thực hiện bằng cách dừng tiến trình tìm kiếm.

8.6. Đối chiếu các phần thực hiện

Bảng 8: Đối chiếu nội dung báo cáo với phần cài đặt UCS.

Nội dung	Cách thực hiện và đối chiếu
Mô hình hóa bài toán	Dùng trạng thái, hàm chuyển và kiểm tra đích của phần lõi.
Thiết kế chi phí	Dùng `StepCost` chung, có chi phí chuyển động và mức phạt theo bản đồ.
Cài đặt tìm kiếm	Hàng đợi theo chi phí tích lũy, loại phần tử cũ, kiểm tra đích khi lấy ra.
Dựng lời giải	Dùng bảng cha và `reconstruct_path`; trả đúng `SolverResult`.
Kiểm chứng kết quả	Chương trình thu thập phát lại đường đi, kiểm tra đích và cộng lại chi phí.
Đo hiệu năng	Đo lặp qua `run_solver()`, tổng hợp trung vị từ các lần chạy.

Bảng này mô tả cách thực hiện, không tự quy đổi thành tỷ lệ hoàn thành. Phần đóng góp cá nhân và đánh giá mức độ hoàn thành cần được nhóm xác nhận.

8.7. Kết luận

UCS tìm lời giải có tổng chi phí nhỏ nhất theo quy tắc `StepCost` trên đồ thị trạng thái do phần lõi cung cấp. Số liệu ghi trong tệp thực nghiệm cho thấy mục tiêu này có thể tạo lời giải khác với tối thiểu số bước, đặc biệt ở màn có tách khối. Đổi lại, thuật toán có thể phải mở rộng nhiều trạng thái vì không sử dụng thông tin ước lượng khoảng cách tới đích.

9. Thuật toán A* và hàm ước lượng

9.1. Phạm vi thực hiện

Cài đặt A*, xây dựng hàm ước lượng (heuristic) và đánh giá lời giải trên đồ thị chuyển trạng thái do phần lõi trò chơi cung cấp.

A* tối thiểu hóa tổng chi phí, không nhất thiết tối thiểu số hành động. Chi phí được tính bằng `StepCost` trong `solvers/common.py`. Hành động `SPACE` cũng được tính vào chuỗi lời giải và có chi phí.

9.1.1. Đầu vào và đầu ra

Đầu vào gồm màn chơi `Level` và trạng thái bắt đầu `GameState`, có thể là trạng thái ban đầu hoặc trạng thái hiện tại của người chơi. `GameState` bất biến và có thể dùng làm khóa của bảng băm. Nó chứa thông tin về khối và mặt nạ bit biểu diễn trạng thái các cầu. Thông tin khối gồm tư thế, trạng thái tách, vị trí hai khối lập phương và khối đang được điều khiển. Thuật toán dùng toàn bộ trạng thái làm khóa tìm kiếm.

Đầu ra là `SolverResult`, gồm chuỗi lời giải, trạng thái thành công, tổng chi phí, số hành động, số lần mở rộng và các số đo thời gian, bộ nhớ. Chuỗi lời giải dùng các ký hiệu `U`, `D`, `L`, `R`, `SPACE`. Nếu trạng thái ban đầu đã thắng, chuỗi rỗng là lời giải hợp lệ có chi phí bằng 0. Nếu hàng đợi tìm kiếm cạn, trường `path` là `None`, `solution_length` bằng -1 và `cost` bằng vô cực.

9.1.2. Ranh giới giữa các thành phần

Bảng 9: Phân chia trách nhiệm trong chương trình hiện tại.

Thành phần	Trách nhiệm
`core/interfaces.py`	Các kiểu trạng thái, màn chơi và kết quả dùng chung.
`core/game_rules.py`	Sinh trạng thái kế tiếp hợp lệ và kiểm tra đích.
`solvers/common.py`	Quy tắc tính chi phí và hàm đo thời gian, bộ nhớ.
`solvers/astar.py`	Quản lý tìm kiếm, hàm ước lượng và dựng lời giải.
`ui/solver_bridge.py`	Chạy thuật toán trong tiến trình riêng và trả kết quả về giao diện.

Phạm vi bảo đảm tính đúng

A* không tự cài lại luật công tắc, cầu hay tách và ghép khối. Tính tối ưu được xét trên đồ thị do phần lõi hiện tại cung cấp, với màn chơi giữ cố định trong mỗi lần giải.

Việc phát lại một lời giải hợp lệ qua phần lõi không thay thế kiểm thử độc lập các luật trò chơi.

9.2. Mô hình chi phí và cấu trúc dữ liệu

9.2.1. Quy tắc tính chi phí dùng chung

Gọi s là trạng thái trước, a là hành động và t là trạng thái sau. Chi phí chuyển trạng thái gồm chi phí chuyển động cơ sở, chi phí bổ sung trên các ô đích và chi phí thay đổi trạng thái cầu:

$$c(s, a, t) = e(s, a, t) + \sum_{x \in C(t)} p(x) + b(s, t). \tag{4}$$

Trong đó $C(t)$ là tập ô mà trạng thái đích chiếm, e là chi phí chuyển động cơ sở, p là chi phí bổ sung trên mỗi ô và b là chi phí thay đổi trạng thái cầu. Đây là mô hình định lượng của chương trình, không phải phép đo năng lượng vật lý. Các hằng số được giữ theo quy tắc tính chi phí được chuyển từ UCS sang `common.py`.

Bảng 10: Chi phí cơ sở của mỗi hành động.

Loại chuyển động	Chi phí cơ sở
Đứng → nằm hoặc kích hoạt tách khối từ tư thế đứng	1.0
Nằm → đứng, khi kết quả chưa tách	1.5
Nằm → nằm	1.2
Di chuyển từ trạng thái đã tách	0.8
`SPACE`: đổi khối lập phương đang điều khiển	0.3

Chi phí bổ sung gồm 0.5 cho mỗi ô dễ vỡ bị chiếm, 0.3 cho mỗi ô cầu bị chiếm, 0.1 cho mỗi ô đích kề khoảng trống theo phép kiểm tra bản đồ, và 0.5 nếu mặt nạ trạng thái cầu thay đổi. Phép kiểm tra kề khoảng trống xét bốn hướng; các ô cầu vẫn được xem là một phần của cấu trúc bản đồ. Chi phí trên các ô đích vẫn được tính cho `SPACE`, đúng theo cài đặt hiện tại. Thuật toán chỉ tính chi phí cho các trạng thái kế tiếp đã được phần lõi chấp nhận, không biến một nước đi thua thành nước đi hợp lệ.

Mọi chi phí bổ sung đều không âm, nên với quy tắc mặc định:

$$c(s, a, t) \geq c_{\min} = 0.3 > 0. \tag{5}$$

`StepCost(level)` tính trước chi phí bổ sung của các ô một lần cho mỗi lần giải. A* có thể nhận hàm tính chi phí tùy chỉnh; nếu không truyền kèm hàm ước lượng riêng, thuật toán mặc định dùng $h = 0$. Cận dưới 0.3 chỉ áp dụng cho quy tắc mặc định, không tự động áp dụng cho quy tắc khác.

9.2.2. Cấu trúc tìm kiếm

- **Hàng đợi ưu tiên:** dùng cấu trúc đống cực tiểu (min-heap), lưu các bộ `(f, counter, g, state)`.
- **Bảng chi phí tốt nhất:** `best_g[state]` lưu chi phí thấp nhất đã biết để tới một trạng thái.
- **Bảng cha:** `parents[state] = (previous_state, action)` dùng để dựng lời giải.
- **Bộ đếm thứ tự:** phân giải trường hợp hòa điểm theo thứ tự đưa vào hàng đợi, tránh phải so sánh trực tiếp các đối tượng trạng thái.

Không dùng riêng tọa độ làm khóa: hai trạng thái cùng vị trí nhưng khác trạng thái cầu hoặc khối đang điều khiển có thể có các bước đi tiếp theo khác nhau. Thuật toán không khóa vĩnh viễn trạng thái đã mở rộng, mà cho phép xét lại khi tìm được đường đến có chi phí thấp hơn.

9.3. Thuật toán A* và cách tích hợp

9.3.1. Hàm ưu tiên

$$f(s) = g(s) + h(s). \tag{6}$$

$g(s)$ là chi phí thực tế của đường từ trạng thái bắt đầu tới s; $h(s)$ là cận dưới của chi phí còn lại. Hàng đợi ưu tiên lấy phần tử có f nhỏ nhất. Nếu hòa f, phần tử được đưa vào trước được xét trước, theo bộ đếm thứ tự và thứ tự sinh trạng thái kế tiếp của phần lõi.

9.3.2. Mã giả

Thuật toán 4: A* trên đồ thị trạng thái với chi phí không đều

Đầu vào: Màn chơi cố định, trạng thái bắt đầu, hàm chi phí và hàm ước lượng không vượt chi phí còn lại.
Đầu ra: Lời giải có chi phí tối ưu, hoặc thông báo không có lời giải khi hàng đợi cạn.

1: Chuẩn bị hàm chi phí; đặt `best_g[start] = 0`, cha của trạng thái bắt đầu là rỗng.
2: Đưa `(h(start), counter, 0, start)` vào hàng đợi.
3: Khi hàng đợi còn phần tử: lấy phần tử có ưu tiên nhỏ nhất.
4: Bỏ qua phần tử nếu `g` không còn bằng `best_g[state]`.
5: Tăng bộ đếm; nếu `is_goal(state, level)`, dựng lời giải và trả chi phí `g`.
6: Xét từng cặp `(action, child)` từ `get_successors(state, level)`.
7: Tính `new_g = g + cost(state, action, child, level)`.
8: Nếu `new_g` nhỏ hơn chi phí tốt nhất của `child`, cập nhật chi phí và cha.
9: Đưa `(new_g + h(child), counter, new_g, child)` vào hàng đợi, kể cả khi trạng thái đó đã mở rộng.
10: Nếu hàng đợi cạn, trả kết quả không có lời giải.

9.3.3. Loại phần tử cũ, mở lại trạng thái và điều kiện tối ưu

`heapq` không hỗ trợ giảm trực tiếp khóa ưu tiên của một phần tử đang có. Vì vậy khi cải thiện g, thuật toán đưa thêm phần tử mới vào hàng đợi và giữ phần tử cũ. Khi phần tử cũ được lấy ra, phép kiểm tra chi phí loại bỏ nó mà không tăng `expanded_nodes`.

Thuật toán chấp nhận đích khi lấy một phần tử còn hiệu lực khỏi hàng đợi, không phải ngay khi vừa sinh ra trạng thái đích. Việc mở lại trạng thái cho phép sử dụng hàm ước lượng chấp nhận được nhưng chưa nhất quán. Với chi phí dương hiện tại, hàm ước lượng không vượt chi phí còn lại và đồ thị trạng thái hữu hạn, thuật toán tìm được lời giải có chi phí tối ưu nếu lời giải tồn tại.

A* kiểm tra chi phí và giá trị ước lượng hữu hạn, chi phí dương, giá trị ước lượng không âm và bằng 0 tại đích. Các phép kiểm tra khi chạy không chứng minh được tính chấp nhận được của hàm ước lượng tùy chỉnh; người thiết kế hàm phải chứng minh điều kiện này.

9.3.4. Cách gọi từ chương trình

```
from core.interfaces import GameState
from levels.level_loader import load_level
from solvers.astar import AStarSolver
from solvers.common import run_solver

level = load_level("level_01")
start = GameState(level.start, level.initial_bridge_mask)
result = run_solver(AStarSolver(), level, start)
```

`solve()` tạo `SolverResult`; `run_solver()` bổ sung số đo thời gian và bộ nhớ. Trong giao diện, bộ kết nối chọn thuật toán và chạy hàm đo đạc trong tiến trình riêng. Luồng giao diện nhận kết quả để phát lại lời giải. Thao tác hủy hiện được thực hiện bằng cách dừng tiến trình tìm kiếm; thuật toán chưa có giới hạn thời gian, số nút hoặc cơ chế nhận yêu cầu hủy nội bộ. Các giao diện dự kiến trong tài liệu thiết kế không được xem là tính năng đã triển khai.

9.4. Thiết kế và chứng minh hàm ước lượng

9.4.1. Vì sao không dùng Manhattan cộng chi phí tư thế

Một lần lăn có thể làm tọa độ neo của khối dịch hơn một ô. Khi tách, khối còn có thể dịch chuyển tức thời tới vị trí xa. Vì vậy khoảng cách Manhattan từ tọa độ neo cộng một điểm cho tư thế nằm không mặc nhiên là cận dưới. Bản này dùng mô hình nới lỏng xét cả các đường dịch chuyển tức thời ngay khi trạng thái hiện tại chưa tách, như trình bày trong `docs/astar-heuristic.md`.

9.4.2. Mô hình điểm và các điểm mốc

Thay khối bằng một điểm trên lưới không giới hạn. Mỗi bước trong mô hình nới lỏng có chi phí 1 và đi tối đa 2 ô Manhattan, kể cả bước chéo (1,1). Mô hình bỏ qua tư thế, khoảng trống, ô dễ vỡ, trạng thái cầu và điều kiện kích hoạt công tắc. Mỗi công tắc tách khối có cạnh dịch chuyển tức thời có hướng tới từng vị trí đích của nó, với chi phí bằng 0.

Khoảng cách di chuyển thông thường giữa hai điểm trong mô hình là:

$$w(p, q) = \left\lceil \frac{|p_1 - q_1| + |p_2 - q_2|}{2} \right\rceil. \tag{7}$$

Chỉ giữ các điểm mốc gồm ô đích, vị trí công tắc tách khối và các vị trí nhận khối sau khi tách. Nối mọi cặp bằng w, thêm cạnh dịch chuyển tức thời với chi phí 0, rồi chạy Dijkstra trên đồ thị đảo từ ô đích. Gọi $d(p)$ là khoảng cách từ điểm p tới ô đích trong mô hình nới lỏng. Với điểm bất kỳ, khoảng cách bằng giá trị nhỏ nhất của chi phí đi tới một điểm mốc cộng khoảng cách còn lại của điểm mốc đó. Cách rút gọn này vẫn xét mọi chuỗi dịch chuyển tức thời: một đường đi hoặc tới thẳng ô đích, hoặc phải tới đầu của cạnh dịch chuyển tức thời đầu tiên.

Với $C(s)$ là tập một hoặc hai ô mà trạng thái thật đang chiếm, đặt:

$$H(s) = \min_{p \in C(s)} d(p), \quad h(s) = 0.3H(s). \tag{8}$$

Khi đã tách, phép lấy giá trị nhỏ nhất xét cả hai khối lập phương, không chỉ khối đang điều khiển. Một trạng thái chưa thắng vẫn có thể có $h = 0$, chẳng hạn khối nằm đè lên ô đích; đây vẫn là cận dưới hợp lệ.

9.4.3. Chứng minh tính nhất quán và tính chấp nhận được

Tính nhất quán yêu cầu $h(s) \le c(s, a, t) + h(t)$ trên mỗi cạnh hợp lệ. Tính chấp nhận được yêu cầu hàm ước lượng không vượt quá chi phí tối ưu còn lại. Xét cạnh thật $s \to t$ có chi phí $c(s, a, t)$. Chọn ô q thuộc $C(t)$ sao cho $d(q) = H(t)$. Có một ô p thuộc $C(s)$ với đường đi tới q trong mô hình nới lỏng có chi phí tối đa 1:

- **Lăn khối:** theo bảng chuyển tư thế, mỗi ô mới cách ít nhất một ô cũ không quá 2 đơn vị Manhattan.
- **Di chuyển khối lập phương:** khối đang điều khiển dịch 1 ô, khối còn lại đứng yên.
- **Đổi điều khiển bằng SPACE:** tập ô không đổi nên có thể mô phỏng bằng đường đi có chi phí 0.
- **Ghép khối:** tập ô sau ghép trùng với tập ô của hai khối lập phương sau chuyển động.

- **Tách và dịch chuyển tức thời:** từ ô cũ tới công tắc trong tối đa một bước nới lỏng, rồi tới vị trí nhận khối với chi phí 0.
- **Cầu và công tắc:** thay đổi mặt nạ trạng thái không ảnh hưởng đến lập luận vì mô hình bỏ các ràng buộc cản đường.

Suy ra $H(s) \le d(p) \le 1 + d(q) = 1 + H(t)$. Vì chi phí thật ít nhất 0.3:

$$h(s) \le 0.3 + h(t) \le c(s, a, t) + h(t). \tag{9}$$

Vậy hàm ước lượng nhất quán theo quy tắc chi phí hiện tại. Mọi chi phí trong mô hình nới lỏng không âm nên $h(s) \ge 0$; tại trạng thái đích, tập ô chỉ chứa ô đích nên giá trị ước lượng bằng 0. Cộng bất đẳng thức nhất quán dọc một đường tới đích cho thấy giá trị ước lượng không lớn hơn chi phí của đường đó. Do đó hàm cũng chấp nhận được.

9.4.4. Chi phí tính toán và giới hạn

Với K điểm mốc, đồ thị đầy đủ có $O(K^2)$ cạnh. Cài đặt Dijkstra với hàng đợi ưu tiên dùng khoảng $O(K^2 \log K)$ thời gian tiền xử lý. Bộ nhớ đệm lưu $O(K)$ khoảng cách cho mỗi cấu hình; hàng đợi tạm có thể chứa $O(K^2)$ phần tử. Mỗi lần đánh giá xét tối đa hai ô đang chiếm và K điểm mốc, cộng chi phí tạo khóa từ các công tắc tách khối. Bộ nhớ đệm LRU giữ tối đa 32 cấu hình ô đích và các cạnh dịch chuyển tức thời.

Giới hạn của hàm ước lượng

Hàm ước lượng bỏ nhiều ràng buộc nên có thể yếu khi lời giải cần phối hợp hai khối lập phương hoặc mở cầu. Không bảo đảm luôn giảm số nút hay thời gian so với $h = 0$. Khi quy tắc chi phí thay đổi, phải cập nhật cận dưới và chứng minh lại; không tự động dùng hệ số 0.3 cho mọi quy tắc tùy chỉnh.

9.5. Thực nghiệm và đánh giá

9.5.1. Mục tiêu và cách bố trí thực nghiệm

Script `typst/collect_all_results.py` chạy A* trên cùng 11 level và từ cùng trạng thái đầu như BFS, DFS và UCS. Mỗi solver được khởi động một lần không ghi nhận, sau đó đo thời gian 3 lần; thứ tự thuật toán được xoay vòng giữa các lượt. Bảng dùng trung vị và độ lệch chuẩn mẫu của thời gian; bộ nhớ đỉnh được đo riêng một lần cho mỗi level.

Mọi đường đi được phát lại bằng `apply_move`; collector kiểm tra tính hợp lệ, độ dài, đích đến và tổng `StepCost`. Chi phí A* được đối chiếu với UCS trên từng level. Môi trường thu thập: Python 3.11.9 trên `Windows-10-10.0.26200-SP0` tổng số lượt đo là 176. CSV tổng hợp, dữ liệu thô và mã băm nguồn nằm trong `data/`.

9.5.2. Cách đo thời gian và bộ nhớ

Thời gian được đo bằng `perf_counter()` quanh `solve()` khi `tracemalloc` tắt; cùng một cách đo được dùng cho cả bốn solver. Trước mỗi lần A* chạy, bộ nhớ đệm heuristic được xóa để các lượt đo có điều kiện tương đương. Thu gom rác và phát lại lời giải nằm ngoài khoảng đo. Lượt đo bộ nhớ chạy riêng với `tracemalloc`.

Bộ nhớ là đỉnh cấp phát do `tracemalloc` ghi nhận, không phải toàn bộ RAM tiến trình. Trường `peak_memory_kb_median` là trung vị số KB qua các lần đo. Độ lệch chuẩn thời gian được báo để thể hiện mức dao động.

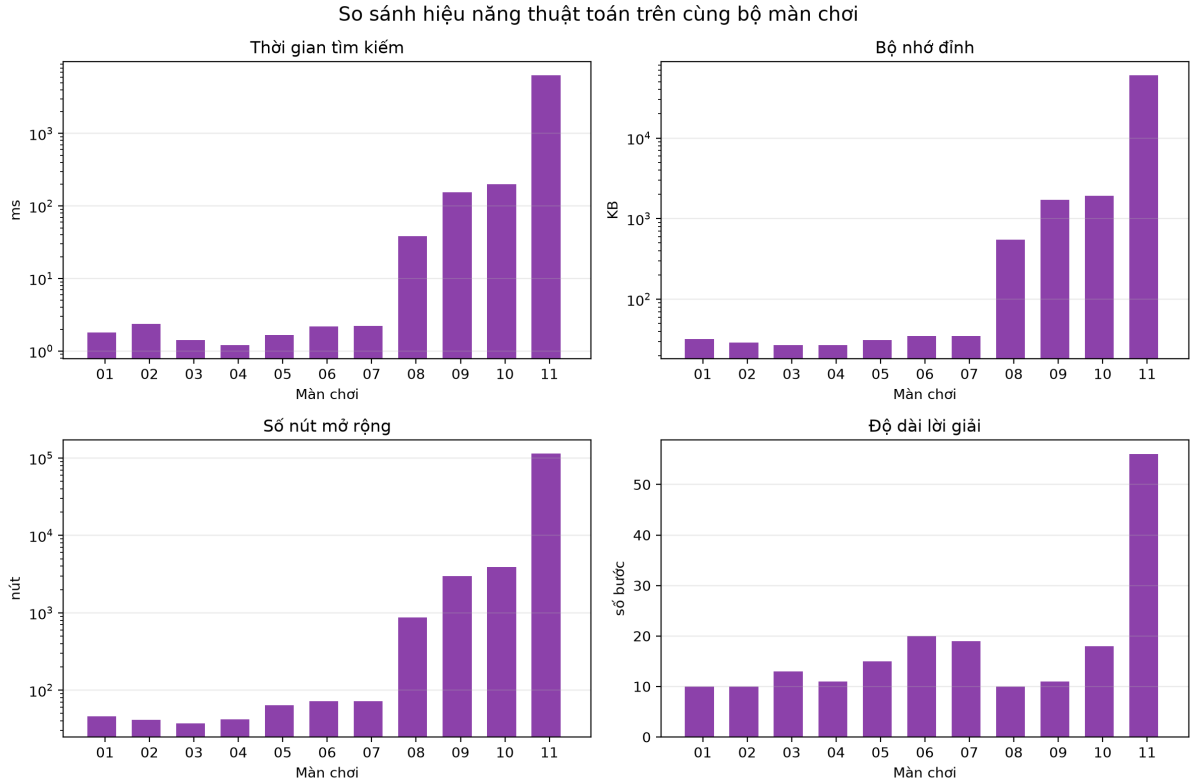
9.5.3. Kiểm chứng lời giải và số nút mở rộng

Ngoài khoảng đo, chương trình xác minh kết quả của cả bốn thuật toán. Mỗi nước đi phải hợp lệ, trạng thái cuối phải thỏa `is_goal`, độ dài phải khớp với số hành động và tổng chi phí UCS/A* phải khớp `StepCost`. Kết quả lời giải và số nút phải ổn định qua các lần đo; sai lệch làm collector dừng và không ghi bộ dữ liệu tổng hợp mới.

Bảng 11: Kết quả A* trên cùng bộ level; thời gian là trung vị qua 3 lượt đo, bộ nhớ đo riêng một lượt.

Màn	Nhóm	Giải được	Bước	Chi phí	Thời gian trung vị (ms)	ĐLC thời gian (ms)	Bộ nhớ đỉnh (KB)	Nút mở rộng
level_01	Cơ bản	Có	10	13.3	1.815	0.506	32	46
level_02	Công tắc/ cầu	Có	10	14.4	2.36	0.416	29	41
level_03	Công tắc/ cầu	Có	13	18.5	1.416	0.262	27	37
level_04	Ô dễ vỡ	Có	11	16.1	1.21	0.023	27	42
level_05	Ô dễ vỡ	Có	15	23.2	1.66	0.171	31	64
level_06	Công tắc/ cầu	Có	20	29	2.184	0.427	35	72
level_07	Công tắc/ cầu	Có	19	27.1	2.222	0.503	35	72
level_08	Kết hợp	Có	10	11	38.247	6.025	549	873
level_09	Kết hợp	Có	11	15.3	154.433	17.626	1701	2965
level_10	Kết hợp	Có	18	24.5	200.79	1.887	1911	3924
level_11	Kết hợp	Có	56	81.16	377.396	178.889	60430	114540

9.5.4. Biểu đồ và nhận xét



Hình 4: Thời gian, bộ nhớ đỉnh, số nút mở rộng và độ dài lời giải của A* theo level.

A* tối ưu tổng `StepCost`, không nhất thiết tối ưu số bước; do đó độ dài lời giải cần được diễn giải cùng chi phí và đối chiếu với BFS/UCS, không dùng riêng để xếp hạng chất lượng. Việc mở rộng ít nút hơn cũng không bảo đảm thời gian thấp hơn vì A* còn tính heuristic và quản lý bộ nhớ đệm.

Phạm vi kết quả

Số liệu mô tả các level trong thư mục `levels/`, trạng thái bắt đầu mặc định, mã nguồn và môi trường ghi trong metadata. Không suy rộng kết quả thành khẳng định A* luôn nhanh hơn hoặc dùng ít bộ nhớ hơn trên mọi màn.

Benchmark không giới hạn thời gian hay số nút; collector dừng nếu solver lỗi, kết quả không ổn định, replay sai hoặc chi phí A* khác UCS. BFS tối ưu số hành động, còn UCS và A* tối ưu tổng `StepCost`; vì vậy không so sánh chất lượng lời giải chỉ bằng số bước.

10. Đánh giá và phân công

10.1. Tổng hợp kết quả

Bảng 12: Mục tiêu tối ưu và phạm vi số liệu của từng thuật toán

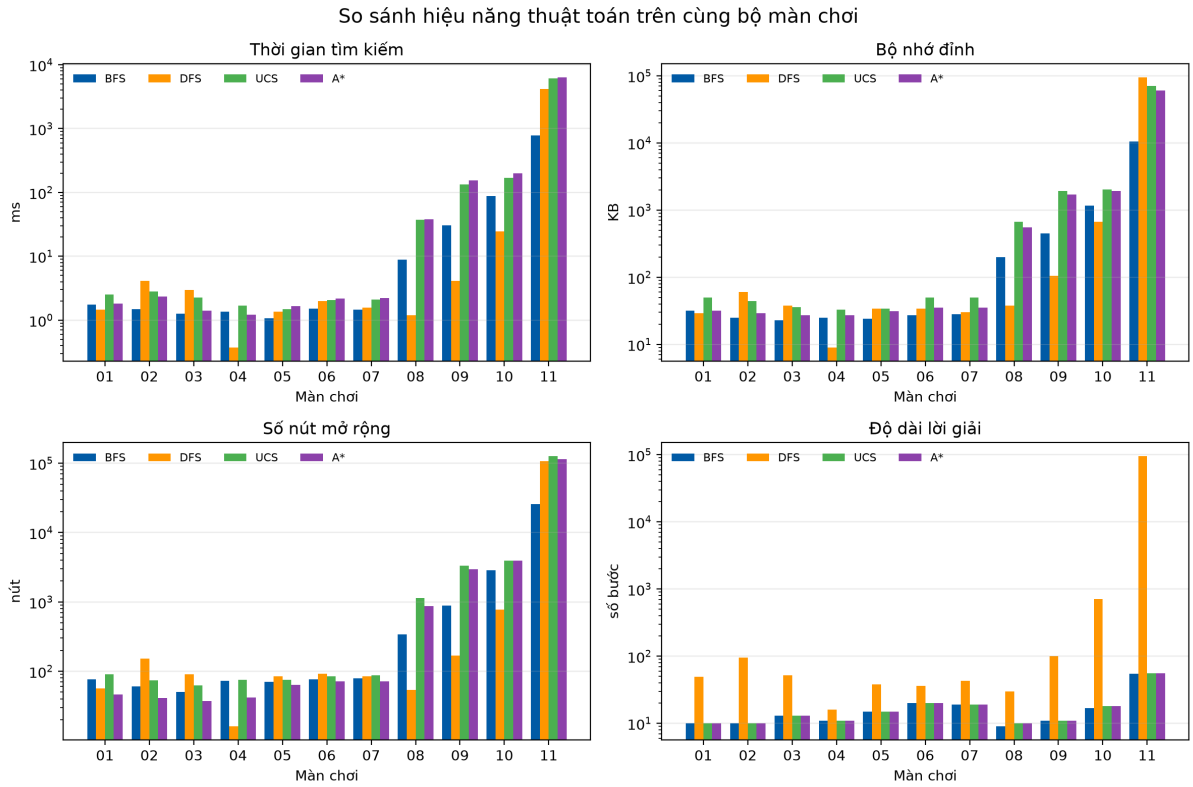
Thuật toán	Mục tiêu tối ưu	Số màn trong bảng riêng	Nội dung thực nghiệm
BFS	Số hành động	11	Thời gian, bộ nhớ đỉnh, nút mở rộng, độ dài lời giải.
DFS	Không bảo đảm tối ưu	11	Thời gian, bộ nhớ đỉnh, nút mở rộng, độ dài lời giải.
UCS	Tổng chi phí `StepCost`	11	Thời gian, bộ nhớ đỉnh, nút mở rộng, độ dài lời giải và chi phí.
A*	Tổng chi phí `StepCost`	11	Thời gian, bộ nhớ đỉnh, nút mở rộng, độ dài lời giải và chi phí.

10.2. So sánh định lượng trên cùng bộ level

Collector đã chạy 4 thuật toán trên 11 level, với 3 lần đo thời gian cho mỗi cặp level/ solver (tổng cộng 132 lần đo thời gian), và một lần đo bộ nhớ riêng cho mỗi cặp (44 lần). Mỗi lời giải được kiểm tra bằng replay; chi phí UCS và A* được đối chiếu trên từng màn.

Bảng 13: Trung vị theo level của bốn chỉ số thực nghiệm; chi tiết từng level nằm trong mục của mỗi thuật toán.

Thuật toán	Giải được	Tổng level	Thời gian trung vị (ms)	Bộ nhớ đỉnh trung vị (KB)	Nút mở rộng trung vị	Độ dài lời giải trung vị
BFS	11	11	1.525	28	77	13
DFS	11	11	2.004	38	90	49
UCS	11	11	2.511	50	88	13
A*	11	11	2.222	35	72	13



Hình 5: So sánh BFS, DFS, UCS và A* theo thời gian, bộ nhớ đỉnh, nút mở rộng và độ dài lời giải trên cùng các level.

Các nhóm test được xác định từ dữ liệu level: cơ bản (không có ô dễ vỡ hay công tắc), ô dễ vỡ, công tắc/cầu, và kết hợp. Biểu đồ dùng thang logarit khi dải số liệu đủ rộng; vì vậy khi đọc cần xem nhãn trục. Trung vị ở bảng tổng hợp là trung vị của các trung vị theo từng level, không phải trung bình có trọng số theo độ khó.

10.3. Kiểm chứng và giới hạn

Các lời giải được kiểm tra bằng cách phát lại hành động qua luật trò chơi, kiểm tra trạng thái đích, và đối chiếu tổng chi phí với `StepCost` đối với UCS/A*. Kiểm thử tự động của dự án bao phủ luật lõi, giao diện và hợp đồng solver; kết quả không thay thế việc kiểm tra độc lập từng cơ chế trong màn chơi.

BFS trả lời bài toán ít bước nhất, còn UCS/A* trả lời bài toán chi phí thấp nhất. Vì vậy không dùng riêng độ dài lời giải để kết luận UCS/A* tốt hơn hoặc kém hơn BFS. DFS là thuật toán đầy đủ trên không gian hữu hạn có tập trạng thái đã thăm, nhưng không bảo đảm tối ưu và hiệu năng phụ thuộc thứ tự successor. Kết luận so sánh chỉ áp dụng cho các level, trạng thái đầu, mã nguồn, số lần lặp và môi trường ghi trong tệp metadata.

10.4. Phân công và tự đánh giá

Danh sách thành viên:

Bảng 14: Bảng xác nhận phân công và tỷ lệ đóng góp của nhóm

STT	Thành viên	Nhiệm vụ được nhóm xác nhận	Tỷ lệ
1	Nguyễn Ngọc Quang — 24120127	Xây dựng giao diện người dùng (GUI), xử lý đầu vào	100%
2	Lương Nhật Tân — 24120134	Cài đặt thuật toán BFS/DFS, đánh giá thuật toán và viết báo cáo về thuật toán BFS/DFS	100%
3	Nguyễn Gia Hưng — 24120185	Cài đặt thuật toán UCS, đánh giá thuật toán và viết báo cáo về thuật toán UCS	100%
4	Võ Nhật Minh — 24120202	Cài đặt thuật toán A*, heuristic, đánh giá thuật toán và viết báo cáo về thuật toán A*	100%
5	Trần Văn Nguyên — 24120205	Xây dựng logic cho game, interface để kết nối GUI với các thuật toán	100%

11. Phụ lục: Công khai việc sử dụng công cụ AI

Trong quá trình thực hiện đồ án, nhóm có sử dụng các công cụ AI tạo sinh để hỗ trợ trao đổi ý tưởng, tìm hiểu và rà soát mã nguồn, cũng như hoàn thiện báo cáo. Các công cụ được sử dụng gồm:

Công cụ	Nhà cung cấp	Mục đích sử dụng
ChatGPT	OpenAI	Hỗ trợ giải thích khái niệm, thảo luận cách trình bày và rà soát nội dung.
Codex	OpenAI	Hỗ trợ đọc, viết và kiểm tra mã nguồn theo yêu cầu của nhóm.
Antigravity	Google	Hỗ trợ trong quá trình phát triển và rà soát dự án.

Các đề xuất do công cụ AI đưa ra không được xem là kết quả đúng mặc nhiên. Nhóm đã xem xét, chỉnh sửa và kiểm chứng mã nguồn, đường đi và kết quả thực nghiệm bằng chương trình, đồng thời chịu trách nhiệm về nội dung cuối cùng của đồ án.

Một số prompt tiêu biểu:

- "Kiểm tra xem các thuật toán tìm kiếm đã đáp ứng yêu cầu của đề bài chưa; nêu rõ căn cứ và điểm cần lưu ý."
- "Phân tích vì sao DFS tìm được đường đi dài và đối chiếu kết quả với BFS trên các level; không thay đổi mã nguồn."
- "Điều chỉnh chức năng giải để thuật toán tìm đường từ trạng thái hiện tại của khối, thay vì luôn quay về trạng thái ban đầu."
- "Dùng cùng bộ level để thu thập số liệu của bốn thuật toán, kiểm tra lời giải và tạo bảng, biểu đồ cho báo cáo."

Tài liệu tham khảo